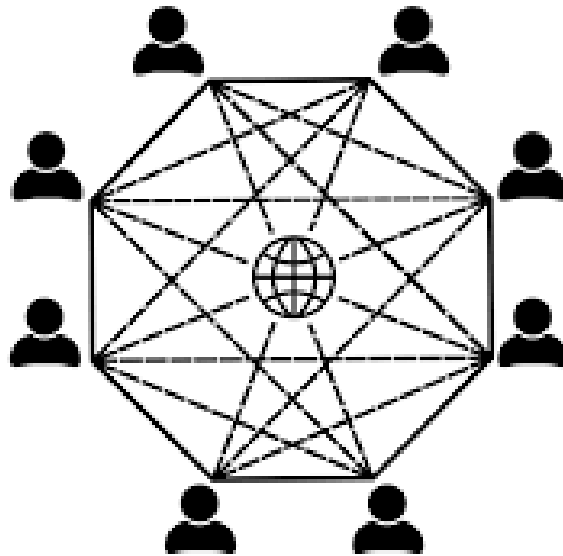
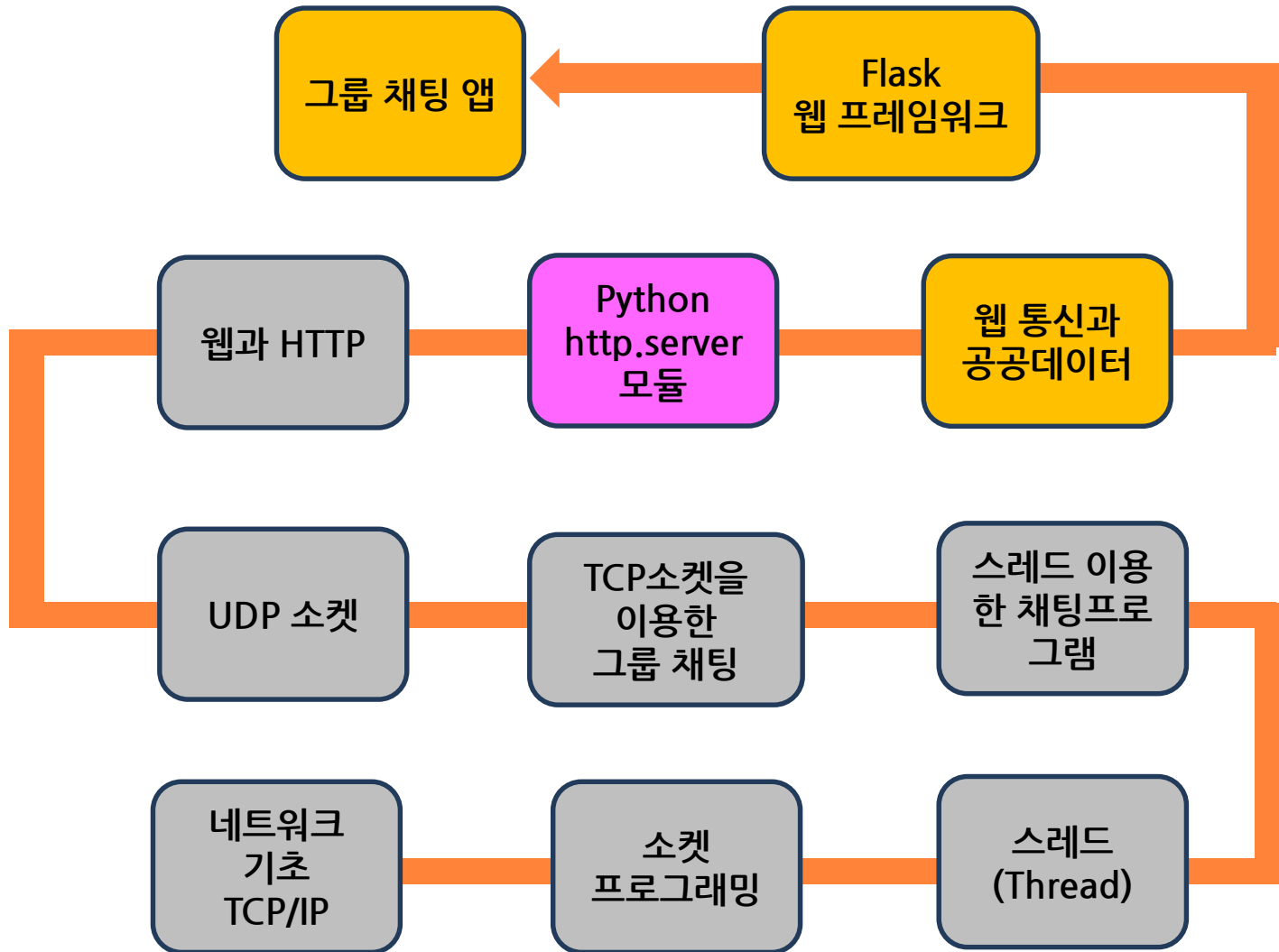


네트워크설계

10주차 - Python http.server 모듈을 이용한 웹 서버구현



10주차는 Python http.server 모듈을 이용한 웹 서버 구현



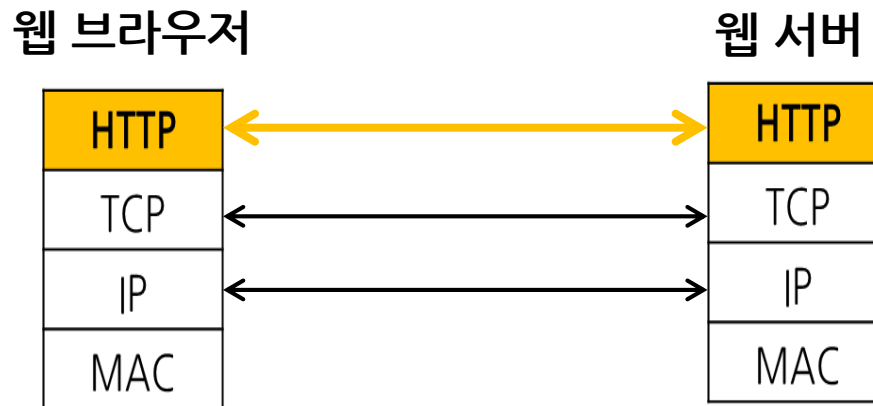
앱 제작 실습 병행

HTTP 핵심정리: 웹 구성요소

■ 웹(Web) 개요

- 웹의 4가지 구성요소

- ① 웹 브라우저와 웹 서버 - 웹 페이지를 주고받는 소프트웨어
- ② HTML - 웹 페이지를 만드는 언어
- ③ URL - 원하는 웹 페이지가 어디에 있는지 알려주는 주소체계
- ④ HTTP - 웹 브라우저와 웹 서버가 통신할 때 사용하는 통신규칙



HTTP 핵심정리: URL 구성

■ URL Uniform Resource Locator

- 웹 주소, 네트워크 상에서 리소스가 어디 있는지 알려주기 위한 주소체계
- URL 형식: **프로토콜 // IP 주소 또는 도메인 주소 : 포트번호 / 자원 위치**

* 아래는 네이버 지도에서 '경기과학기술대학교' 검색시의 URL이다

[https:// map.naver.com \(:443\) /p/search/경기과학기술대학교 ?c=15.00,0.0,2,dh](https://map.naver.com(:443)/p/search/경기과학기술대학교?c=15.00,0.0,2,dh)
Protocol host port path query

- ① Protocol(프로토콜): 어떤 프로토콜을 사용하는지를 알려주는 역할 (http 또는 https)
- ② Host(호스트): 해당 웹사이트의 도메인 주소(네임 서버)
- ③ Port(포트) : URL엔 생략되어 있음 (http는 80, https는 443을 기본포트)
- ④ Path(경로): 해당 웹사이트의 더 자세한 경로를 나타내는 역할
- ⑤ Query(쿼리) : URL에서 "?" 뒤에 조건에 맞는 파라미터가 따라오며
입력한 값에 따라 다른 결과를 보여줘야 할 때 쿼리 문자열을 사용

HTTP 핵심정리: HTTP란?

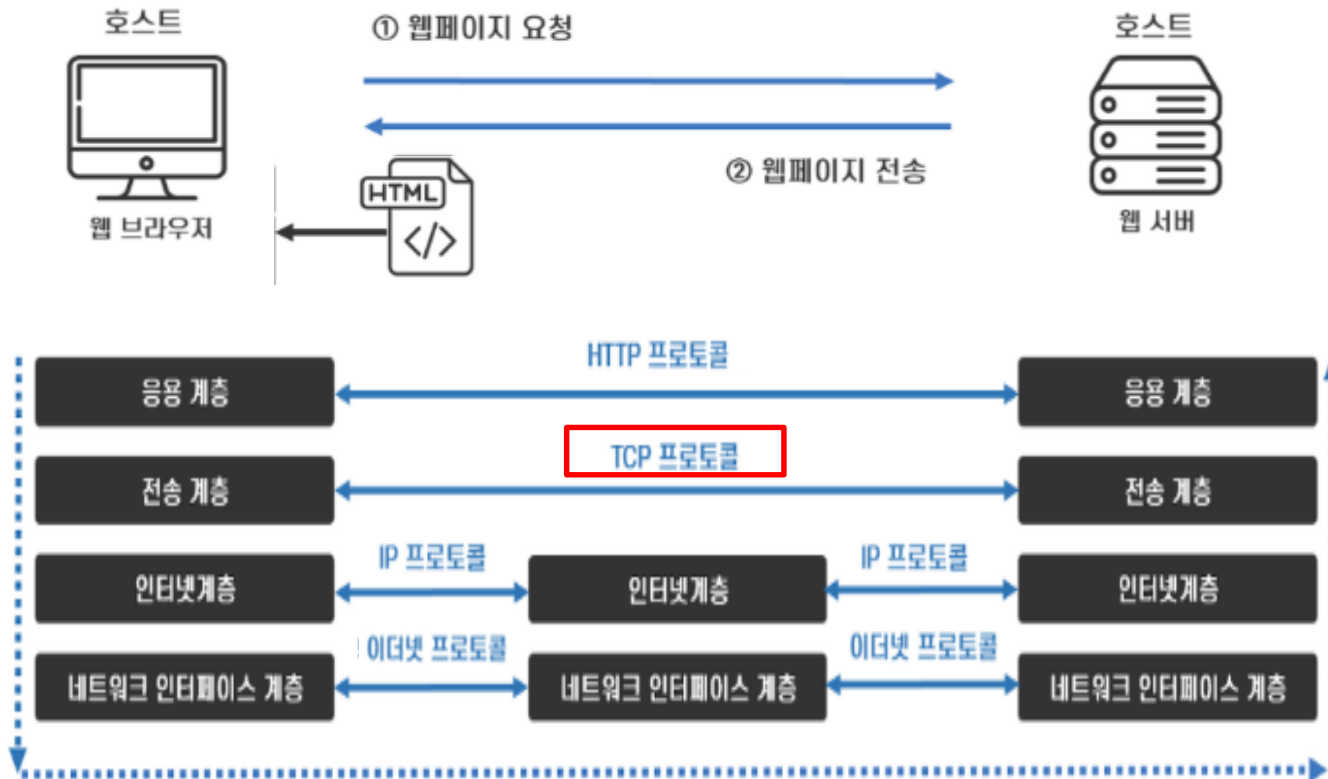
■ HTTP 개요

- 서버와 클라이언트간 HTML 문서를 주고 받기 위한 통신 규칙으로

웹 브라우저의 요청(Request) 메시지와 웹 서버의 응답(Response) 메시지의 구조를 정의(규정)

- HTTP (HyperText Transfer Protocol)과

HTTPS (HyperText Transfer Protocol over Secure Socket Layer): 보안이 강화된 HTTP

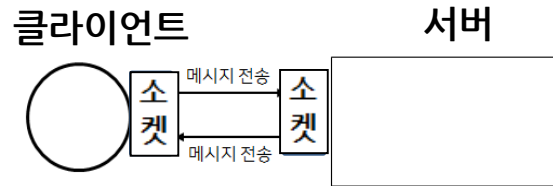


HTTP 핵심정리: 소켓과 HTTP 차이

■ HTTP 개요

◦ HTTP 통신과 소켓 통신의 차이

소켓 통신

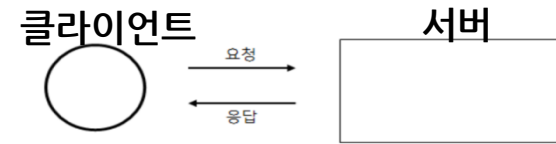


- 서버와 클라이언트 양방향 통신

:클라이언트와 서버 양쪽에서 서로에게 데이터를 전달하는 방식의 양방향 통신

- 서버나 클라이언트에서 강제로 접속을 해제할 때 까지는 계속 **연결을 유지하는 방식**
: 클라이언트 수가 늘어날 수록 서버의 부하 증가 → 접속할 수 있는 클라이언트 수 제한

HTTP 통신



- 클라이언트 요청이 있을 때 서버가 응답하는 방식. **단방향 통신**

:클라이언트에서 서버로 요청을 보내고 서버는 응답하는 방식

- JSON, Image, HTML 파일 등 다양한 파일을 전송 받을 수 있음
- 응답을 받은 후 연결이 끊어지는 것이 기본 동작(HTTP 1.0)
- 성능 상으로 필요하다면 Keep Alive 옵션을 주어 일정 시간 동안 연결을 유지하는 것이 가능 (HTTP 1.1 이상)

“HTTP 통신은 소켓통신의 일종”

HTTP 핵심정리: HTTP 특징

■ HTTP 개요

○ HTTP 특징

① 서버와 클라이언트 구조

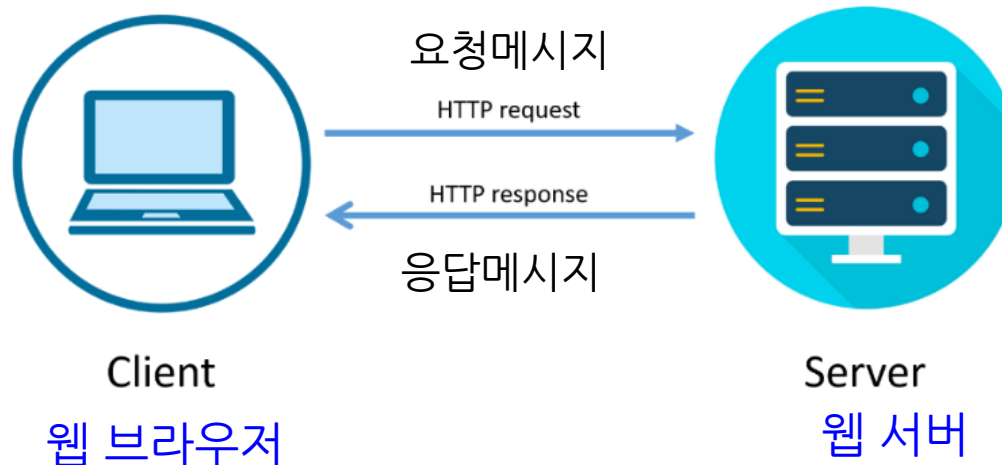
② 무상태 프로토콜(Stateless)

- 서버는 클라이언트 요청정보에 대해서만 응답하며, 이전 요청과는 무관

③ 비연결성 (Connectionless) 프로토콜

- HTTP는 기본적으로 연결을 유지하지 않는 모델
단, HTTP 1.1부터 HTTP 지속 연결 가능 (Persistent Connections)

<https://comic.naver.com/index>



HTTP 핵심정리: HTTP 요청메시지 구조

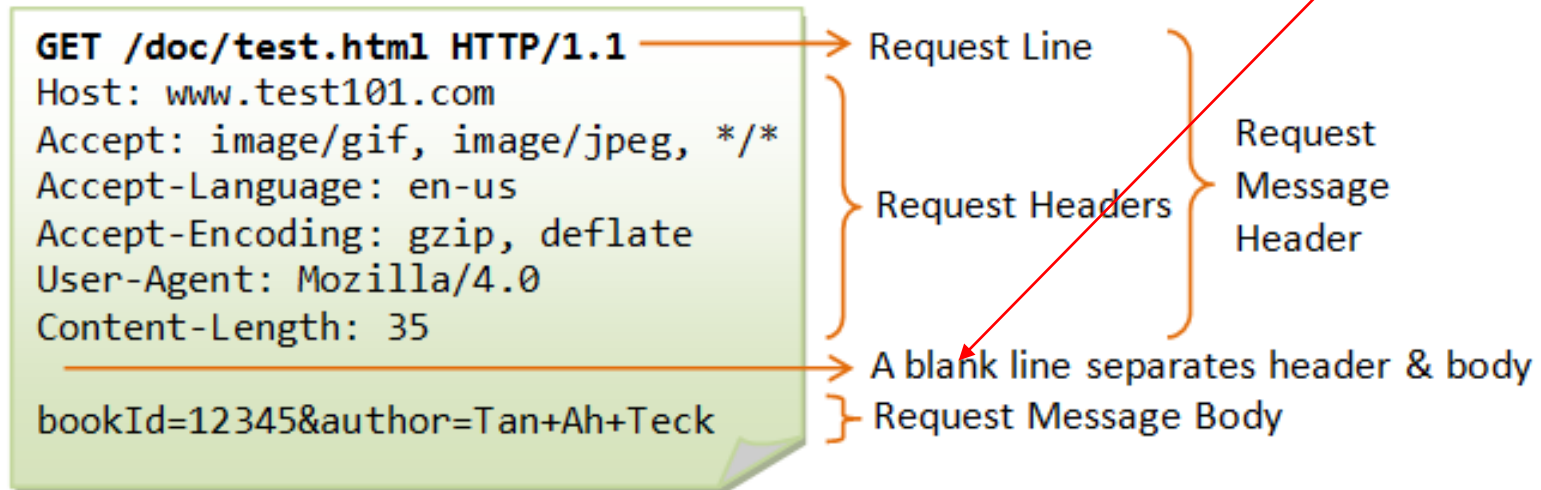
■ HTTP 요청 메시지 (Request Message) 구조

○ HTTP Request Message의 구성

- Request Line
- Headers
- blank line (공백)
- Body

※ 전송하는 데이터가 없다면 body 부분은 비어있음
GET 요청 시에는 Body는 비어있고
POST 요청일 경우 Body에 정보가 포함되어 있음

헤더와 바디를 구분하기 위해
헤더와 바디 사이에
blank line(공백)이 있음
→ **CRLF(\r\n)** : 공백라인

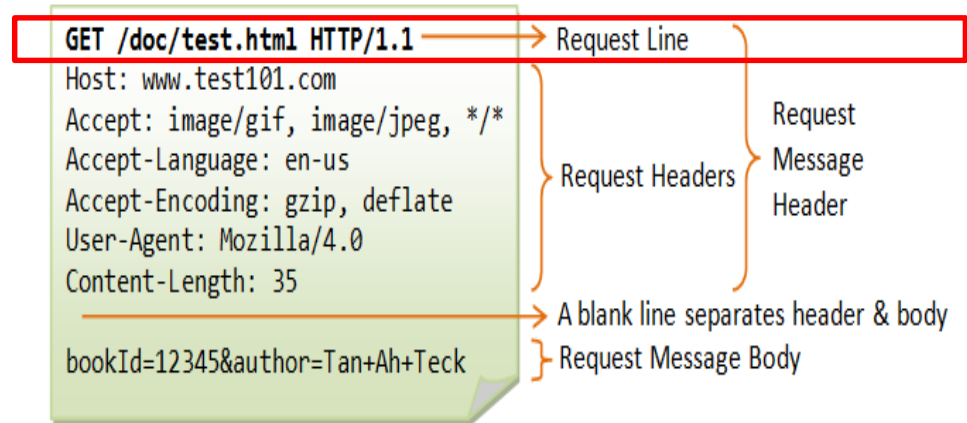


HTTP 핵심정리: HTTP 요청메시지 구조

■ HTTP Request Message 구조

Request line

- HTTP Request Message의 시작 라인
- 3가지 부분으로 구성
 - ① HTTP method
 - ② URL
 - ③ HTTP version



GET /doc/test.html HTTP/1.1
[HTTP Method] [URL] [HTTP version]

- ① **HTTP Method** : 클라이언트가 웹 서버에게 요청하는 목적을 알리는 수단
종류) GET 방식, POST 방식등
 - GET은 서버로 정보를 요청하는 목적 (“서버야 ~~ 좀 줘!”)
 - POST는 서버로 정보를 전달하는 목적 (“서버야 ~~ 줄 테니 처리해 줘”)
- ② **URL** : 요청 자원 명시
- ③ **HTTP version**: HTTP version을 명시(HTTP 1.0, 1.1, 2.0 등)

HTTP 핵심정리: HTTP 응답메시지 구조

■ HTTP 응답메시지(Response Message) 구조

○ HTTP Response Message의 구성

- Status Line
- Headers
- Blank line(공백)
- Body

헤더와 바디를 구분하기 위해
헤더와 바디 사이에
blank line(공백)이 있음
→ **CRLF(\r\n)** : 공백라인



HTTP 핵심정리: HTTP 응답메시지 구조

■ HTTP Response Message 구조

Status line

- HTTP Response의 상태를 나타내는 부분
- 3가지 부분으로 구성

- ① HTTP version
- ② Status Code
- ③ Status Text



HTTP/1.1 200 OK

[HTTP version] [Status Code] [Status Text]

HTTP Status Codes		
Level 200	Level 400	Level 500
200: OK 201: Created 202: Accepted 203: Non-Authoritative Information 204: No content	400: Bad Request 401: Unauthorized 403: Forbidden 404: Not Found 409: Conflict	500: Internal Server Error 501: Not Implemented 502: Bad Gateway 503: Service Unavailable 504: Gateway Timeout 599: Network Timeout
성공적 처리	클라이언트측 오류	서버측 오류

추가적인 페이지 이동필요

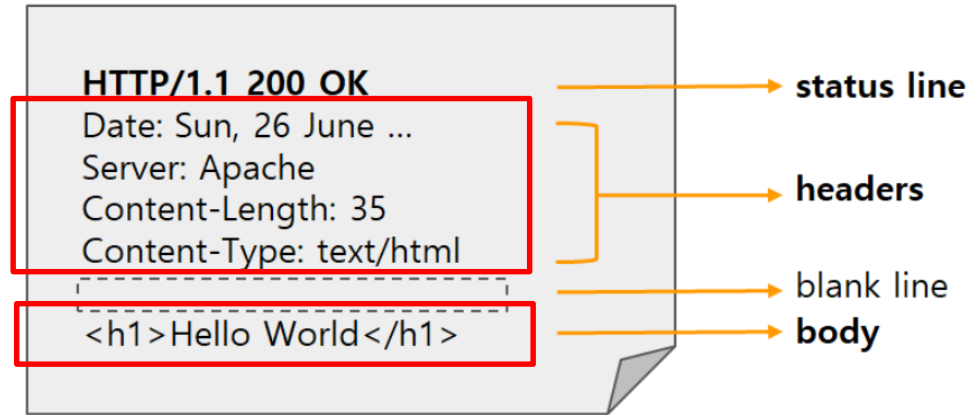
3xx Redirection	
301	Moved Permanently
302	Found
303	See Other
304	Not Modified

캐시관련: 바뀐 게 없으니
캐시 데이터 그냥 사용해!

HTTP 핵심정리: HTTP 응답메시지 구조

■ HTTP Response Message 구조

Headers



- Response의 headers와 거의 유사
- 네트워크상에서 데이터를 보낼 때 데이터의 형식을 나타내는 표준방식인 MIME 타입 사용
 - **MIME* *Multipurpose Internet Mail Extensions* : 미디어 타입 (주타입/서브타입)
 - 서버는 모든 HTTP 객체에 **MIME** 타입을 붙임
- **Content-type**: 보내는 자원의 형식을 명시하기 위해 헤더에 실리는 정보 (MIME 타입)
예) Content-Type: text/html - text 중 html 파일
- Content-Length : body의 길이(바이트 단위)

유형	MIME Type	주요 용도
문서	text/html , text/plain	웹 페이지 및 일반 텍스트
스타일	text/css	디자인 (CSS)
스크립트	text/javascript	동적 기능 (JS)
데이터	application/json	API 데이터 교환
이미지	image/png	투명 지원 이미지
비디오	video/mp4	표준 영상 파일
기타	application/pdf	문서 배포

Body

- 보통은 클라이언트가 요청한 HTML 문서가 들어감

HTTP 핵심정리: 웹 서버와 웹 브라우저 기본 동작

■ 실습1) 웹서버와 웹 브라우저 통신 실습

웹 브라우저

http://localhost.com:8080

Simple HTTP Server

This is index page

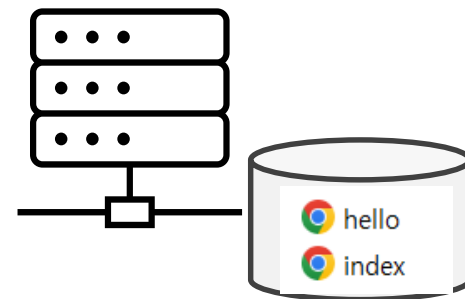
GET /index.html HTTP/1.1



HTTP/1.1 200 OK

body에 index.html 담아서 전송

웹 서버(내 컴퓨터)



http://localhost.com:8080/hello.html

Simple HTTP Server

This is hello page

GET heool.html HTTP/1.1



HTTP/1.1 200 OK

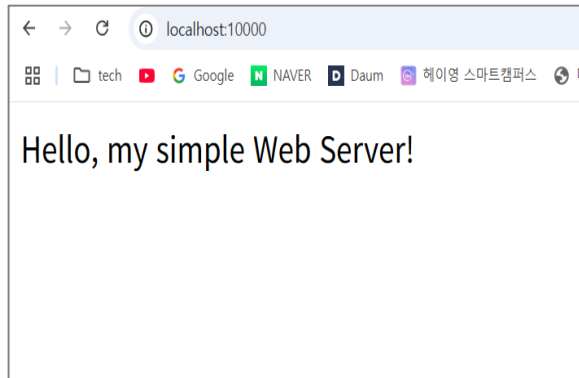
body에 hello.html 담아서 전송

```
PS C:\Jupyter_Project\NetworkDesign\week9> python -m http.server 8080
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
::1 - - [27/Apr/2025 15:15:35] "GET / HTTP/1.1" 200 -
::1 - - [27/Apr/2025 15:23:47] "GET /hello.html HTTP/1.1" 200 -
```

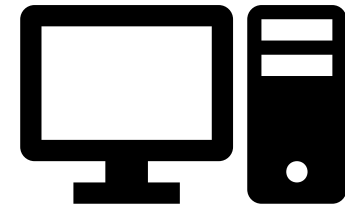
HTTP 핵심정리: 파이썬 코드만으로 간단한 웹 서버 구현

- 실습2) 간단한 HTTP 서버를 제작하여 웹 브라우저와 연동하기

HTTP 클라이언트
(웹 브라우저)



Simple HTTP 서버
(simpleHttpServer.py)



Response message 구조

```
HTTP/1.1 200 OK
Content-length: body의 길이(바이트)
Content-type: text/html

body
```

```
body = "<html><body>Hello, my simple Web Server!</body></html>"
```

HTTP 핵심정리: 파이썬 코드만으로 간단한 웹 서버 구현

■ 간단한 HTTP 서버 프로그램

파일명: simpleHttpServer.py

```
1  # Only python으로 simple HTTP server 제작하기
2
3  import socket
4
5  IP = 'localhost'
6  PORT = 10000
7  address = (IP, PORT)
8
9  print("++ Simple HTTP server")
10 # TCP 소켓 생성
11 server = socket.socket()
12 # (부가코드) wiError 10048 에러해결을 위한 코드
13 server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
14 # bind()
15 server.bind(address)
16 # listen()
17 server.listen()
18 print("++ HTTP Server wait.....")
19
```

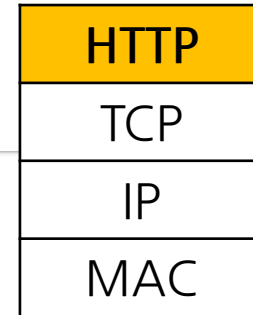
HTTP
TCP
IP
MAC

HTTP 핵심정리: 파이썬 코드만으로 간단한 웹 서버 구현

■ HTTP 서버 프로그램

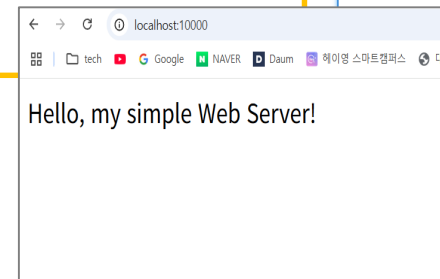
- 웹 브라우저로 요청메시지 받고, 응답메시지 보내기

```
19 while True:
20     # accept()
21     conn, addr = server.accept()
22     print(f"{addr} is connected")
23     # 웹브라우저 요청메시지 수신
24     req_message = conn.recv(1024).decode('utf-8')
25     print("Request message received")
26     lines = req_message.splitlines() # 줄단위로 분리
27     for line in lines:
28         print(line)
29     # HTTP 응답메시지 보내기
30     # 응답메시지 구조 = status_line + headers + CRLF(\r\n) + body
31     body = "<html><body>Hello, my simple HTTP Server!</body></html>"
32     status_line = "HTTP/1.0 200 OK\r\n"
33     headers = f"Content-type: text/html\r\nContent-Length: {len(body)}\r\n"
34     CRLF = "\r\n" # blank line(공백라인)
35     res_message = status_line + headers + CRLF + body
36     conn.sendall(res_message.encode('utf-8'))
37     print("++ Response message Send")
38     conn.close()
39 server.close()
40 print("++ HTTP Server End")
```



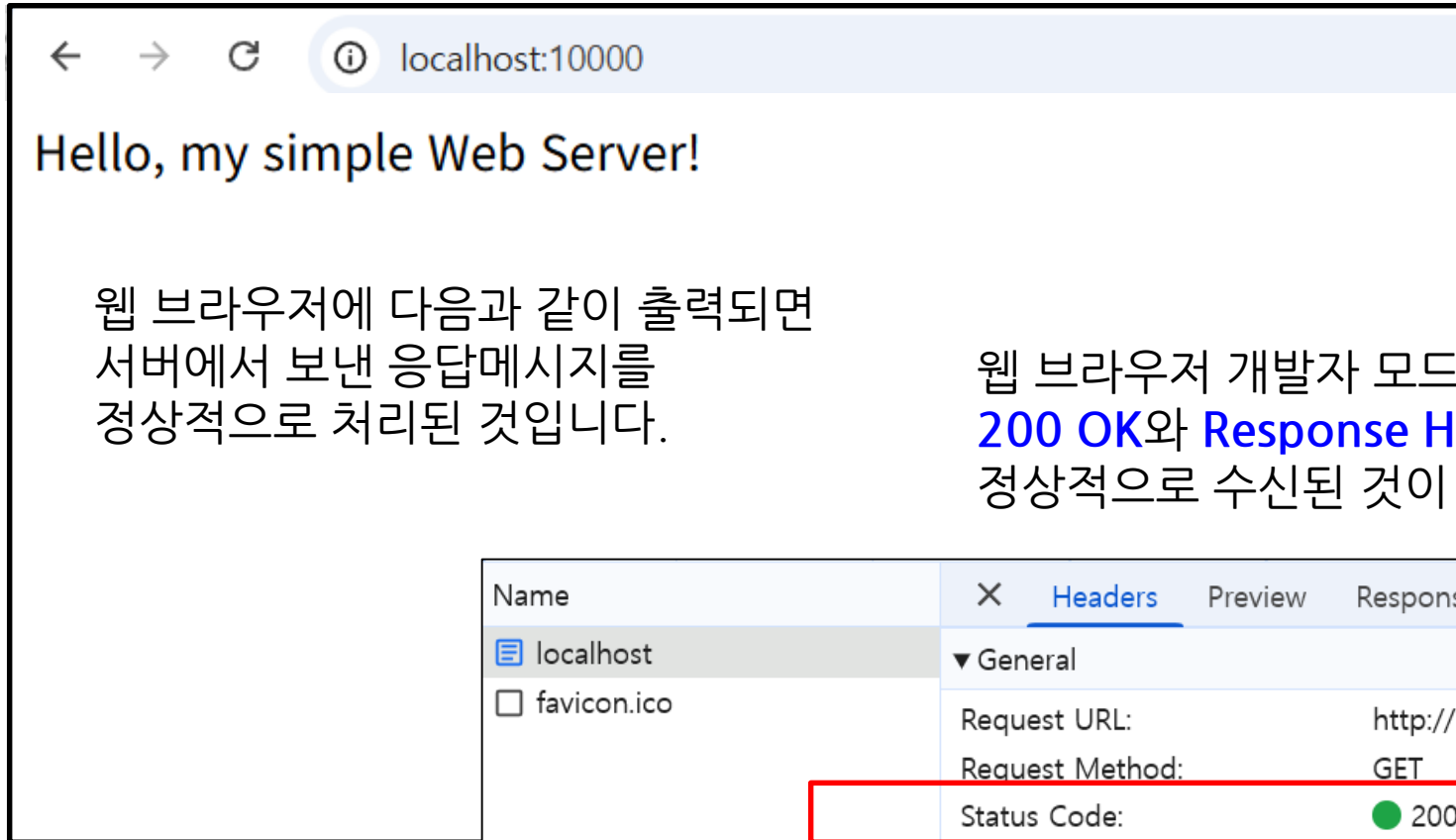
※ TCP 소켓 위에서,
HTTP 응답메시지 형식에 맞게
메시지 제작해서 브라우저로 전송

- Status Line
- Headers
- Blank line(공백)
- Body



HTTP 핵심정리: 파이썬 코드만으로 간단한 웹 서버 구현

■ 웹 브라우저에서 서버에 접속해 보세요



웹 브라우저 개발자 모드(F12)에서 **200 OK**와 **Response Headers** 정보가 정상적으로 수신된 것이 보입니다.

Name	×	Headers	Preview	Response	Initiator	Timing
localhost		▼ General				
☐ favicon.ico		Request URL:	http://localhost:10000/			
		Request Method:	GET			
		Status Code:	● 200 OK			
		Remote Address:	127.0.0.1:10000			
		Referrer Policy:	strict-origin-when-cross-origin			
		▼ Response Headers	☐ Raw			
		Content-Length:	54			
		Content-Type:	text/html			

1. http.server 모듈 개요

■ Python http.server 모듈

http.server — HTTP servers



- 파이썬으로 HTTP 서버를 구현하기 위한 기본 모듈

http.server는 별도의 외부 라이브러리 없이 파이썬 만으로 간단한 웹 서버를 쉽게 구축할 수 있게 해주는 내장 모듈

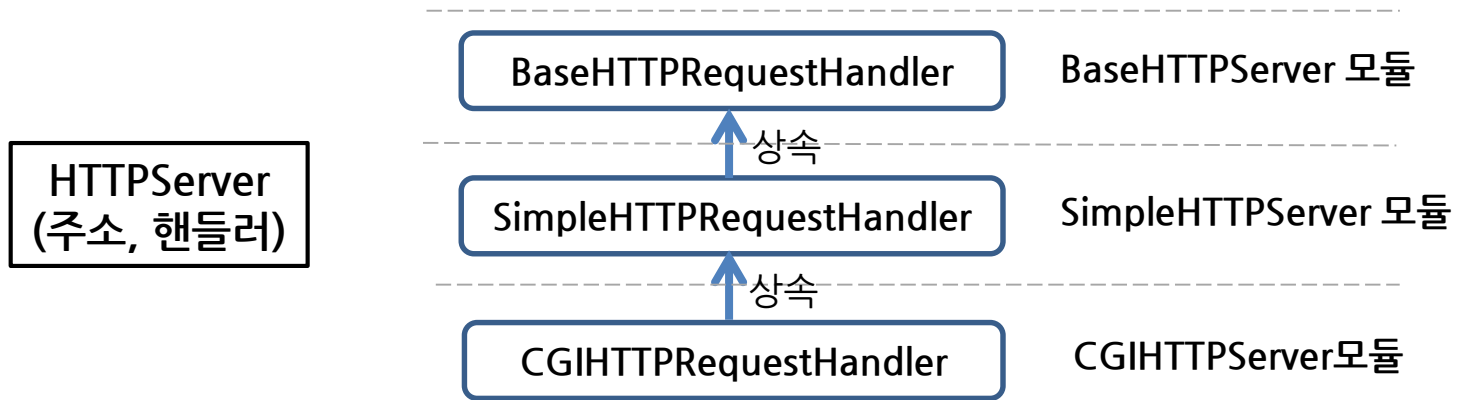
- 웹서버를 만드는데 필요한 라이브러리를 http.server 모듈에서 정의
- 핵심 클래스
 - 1) **HTTPServer 클래스**: 웹 서버 객체를 만들기 위한 클래스
 - 객체 생성시 서버 주소정보(IP와 PORT)와 핸들러 클래스 반드시 필요
 - 2) **핸들러 클래스** : 어떤 요청이 들어올 때 어떻게 응답할지 정의
 - BaseHTTPRequestHandler 또는 SimpleHTTPRequestHandler 클래스

→ HTTPServer 클래스와 핸들러 클래스를 사용하여 직접 서버 동작을 사용자 정의할 수 있음

1. http.server 모듈 개요

■ http.server 모듈

- 파이썬은 HTTP 서버를 구현하기 위한 기본 모듈을 제공 → http.server
- 간단한 HTTP 서버(웹 서버)를 구현하고자 사용하는 모듈



- http.server가 제공하는 핸들러의 종류(어느 정도 구현이 되어 있는가의 차이)

종류	핸들러 클래스	특징
가장 기본 기본적인 프레임만 제공, 사용자가 로직 직접 추가	BaseHTTPRequestHandler	- 어떤 명령 메소드도 구현되지 않은 기본 서버 *메소드 오버라이딩(Method Overriding) 사용 :부모 클래스의 메소드를 상속받아 자식 클래스에서 재정의하는 방식
정적 파일 서버 GET은 구현되어 있음	SimpleHTTPRequestHandler	- BaseHTTPServer의 서브클래스 - GET, HEAD method 구현된 서버
스크립트 실행용 GET, POST 구현	CGIHTTPRequestHandler	- SimpleHTTPServer의 서브클래스 - POST method구현

1. http.server 모듈 개요

■ HTTP 요청 메소드

메소드	특징
GET	- 리소스를 얻어 오기 위한 요청 - 바디 사용 없이 쿼리 스트링으로 요청 데이터 전달
POST	- 새로운 리소스를 생성하기 위한 요청 - 바디를 사용해 요청 데이터 전달
PUT	- 리소스 수정 요청(원래 자원에서 새로운 정보 추가, 삭제하기 위한 요청) - 바디를 사용해 데이터 전달
DELETE	- 리소스를 삭제하기 위한 요청 - 바디 사용 없이 쿼리 스트링으로 데이터 전달

2. http.server 모듈을 이용한 간단한 HTTP 서버 프로그래밍

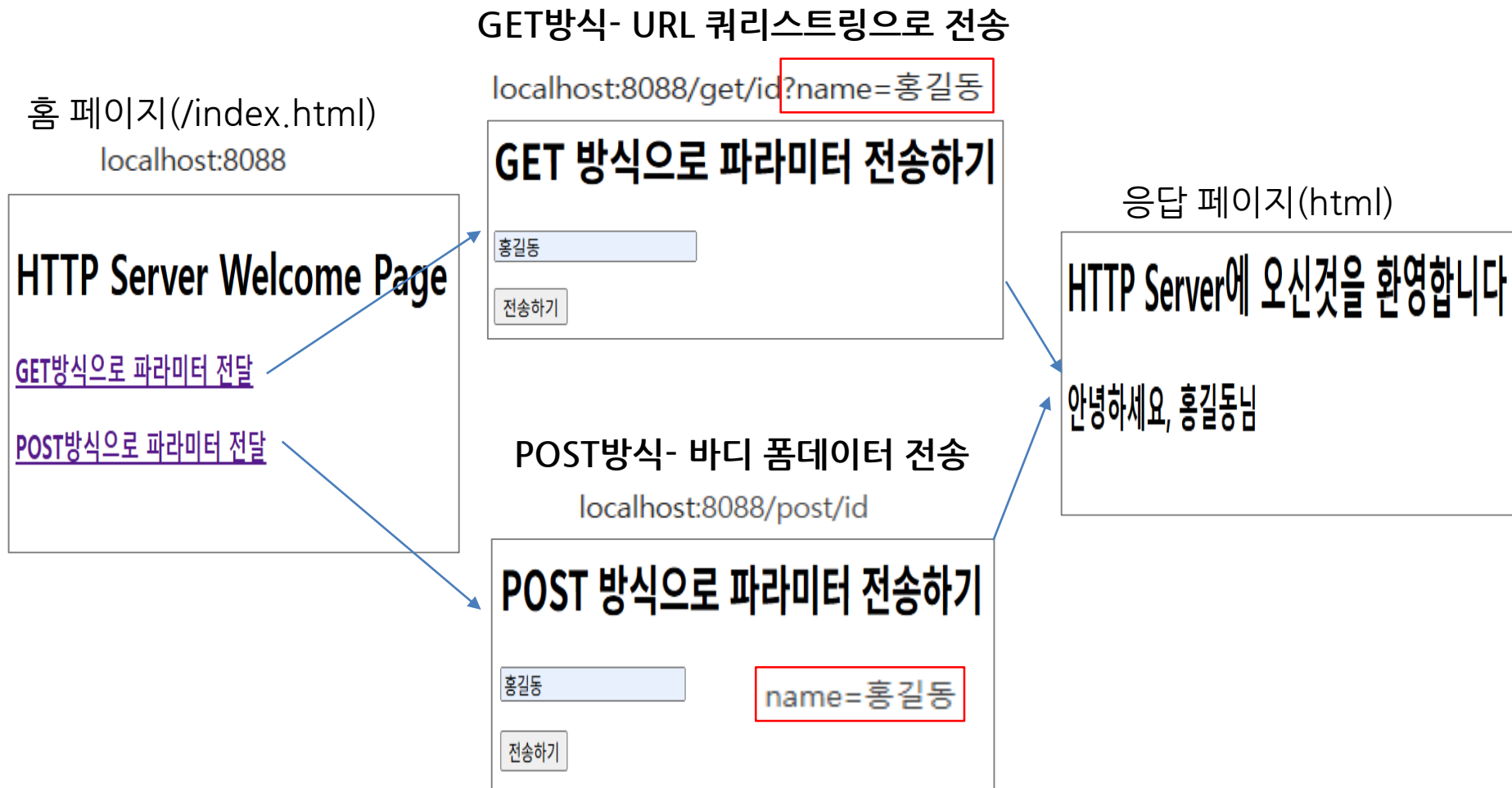
■ 실습1) http.server 모듈의 기본 사용법 (파일명: httpServer.py)

```
1 # http.server 모듈을 이용한 간단한 웹 서버 구현
2 from http.server import HTTPServer, BaseHTTPRequestHandler
3
4 # 웹 페이지 작성(index.html)
5 html_index = '''
6     <html>
7     <body>
8         <h1>This is simple my HTTP Server using http.server</h1>
9     </body>
10 </html>
11 '''
12
13 # base 핸들러 상속받아 자체 핸들러 구현: 클라이언트 요청에 대한 처리 담당
14 class myHandler(BaseHTTPRequestHandler):
15     # 서버로 들어오는 모든 URL의 GET 요청에 대해 동일한 웹 페이지 전송
16     def do_GET(self): # 빈 로직을 상속받아 메소드 오버라이딩으로 자체 구현
17         print("GET 요청들어옴")
18         self.send_response(200) # status line(HTTP/1.1 200 OK)
19         self.send_header("Content-type", "text/html; charset=utf-8") # 응답바디의 타입지정
20         self.end_headers() # 더 이상 헤더없음. blank line(\r\n) 추가
21         # 클라이언트로 전송할 응답 바디 전송
22         self.wfile.write(html_index.encode('utf-8'))
23
24 if __name__ == "__main__":
25     address = ('localhost', 8080)
26     # HTTP 서버 생성: HTTPServer(주소, 핸들러)
27     server = HTTPServer(address, myHandler)
28     print(f"웹 서버가 {address}로 서비스 되고 있습니다")
29     server.serve_forever() # 서버시작(looping)
```

This is simple my HTTP Server using http.server

3. http.server 모듈을 이용한 GET, POST 메소드 구현

- 실습2) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍
 - GET 방식과 POST 방식으로 파라미터(폼데이터)를 받아 처리하기



3. http.server 모듈을 이용한 GET, POST 메소드 구현

■ 실습2) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍

파일명: httpServer_method.py

- 홈 페이지(첫 화면) html 작성

```
1 # http.server 모듈을 이용한 웹 서버 구현(GET, POST 방식처리)
2 from http.server import HTTPServer, BaseHTTPRequestHandler
3
4 # URL 구성요소별 파싱하는 모듈(URL:프로토콜, 도메인, 파라미터(쿼리))
5 from urllib import parse
6
7 # 웹 서버 홈 페이지 제작(첫 화면)
8 html_index = '''
9     <!doctype html>
10     <html>
11         <body>
12             <h1>HTTP Web Server Welcome Page</h1>
13             <h3><a href="/get">GET 방식으로 파라미터 전달</a></h3>
14             <h3><a href="/post">POST 방식으로 파라미터 전달</a></h3>
15         </body>
16     </html>
17 '''
```

HTTP Server Welcome Page

[GET방식으로 파라미터 전달](#)

[POST방식으로 파라미터 전달](#)

3. http.server 모듈을 이용한 GET, POST 메소드 구현

■ 실습2) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍

- GET 요청과 POST 요청을 받을 웹 페이지 작성

```
18 # GET 요청으로 파라미터를 전송하는 웹 페이지 제작
```

```
19 html_get = '''
```

```
20     <!doctype html>
```

```
21     <html>
```

```
22         <body>
```

```
23             <h1>GET 방식으로 파라미터 전송하기</h1>
```

```
24             <form method="GET" action="/get/id">
```

```
25                 <p><input type="text" name="name" placeholder="Enter your name"></p>
```

```
26                 <p><input type="submit" value="전송하기"></p>
```

```
27             </form>
```

```
28         </body>
```

```
29     </html>
```

```
30 '''
```

```
31 # POST 요청으로 파라미터를 전송하는 웹 페이지 제작
```

```
32 html_post = '''
```

```
33     <!doctype html>
```

```
34     <html>
```

```
35         <body>
```

```
36             <h1>POST 방식으로 파라미터 전송하기</h1>
```

```
37             <form method="POST" action="/post/id">
```

```
38                 <p><input type="text" name="name" placeholder="Enter your name"></p>
```

```
39                 <p><input type="submit" value="전송하기"></p>
```

```
40             </form>
```

```
41         </body>
```

```
42     </html>
```

```
43 '''
```

```
44
```

GET 방식으로 파라미터 전송하기

POST 방식으로 파라미터 전송하기

3. http.server 모듈을 이용한 GET, POST 메소드 구현

■ 실습2) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍

- 핸들러 작성 (서로 다른 경로의 GET 요청에 대한 처리 구현)

```
45 # base 핸들러 상속받아 자체 핸들러 제작
46 class myHandler(BaseHTTPRequestHandler):
47     def do_GET(self): # 빈 로직 상속받아 메소드 오버라이딩으로 자체 구현
48         print("GET 요청 들어옴")
49         # 경로의 끝이 루트('/') 인 경우 (기본요청)
50         if self.path.endswith('/'):
51             self.send_response(200) # status line: HTTP/1.1 200 OK
52             self.send_header("Content-type", "text/html; charset=utf-8") #바디 type 지정
53             self.end_headers() # 더이상 헤더없음. blnak line 추가
54             # 응답메시지의 바디 전송
55             self.wfile.write(html_index.encode('utf-8'))
56
57         # 경로의 끝이 ('/get') 인 경우
58         if self.path.endswith('/get'):
59             self.send_response(200) # status line: HTTP/1.1 200 OK
60             self.send_header("Content-type", "text/html; charset=utf-8") #바디 type 지정
61             self.end_headers() # 더이상 헤더없음. blnak line 추가
62             # 응답메시지의 바디 전송
63             self.wfile.write(html_get.encode('utf-8'))
64
65         # 경로의 끝이 ('/post') 인 경우
66         if self.path.endswith('/post'):
67             self.send_response(200) # status line: HTTP/1.1 200 OK
68             self.send_header("Content-type", "text/html; charset=utf-8") #바디 type 지정
69             self.end_headers() # 더이상 헤더없음. blnak line 추가
70             # 응답메시지의 바디 전송
71             self.wfile.write(html_post.encode('utf-8'))
72
```

3. http.server 모듈을 이용한 GET, POST 메소드 구현

■ 실습2) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍

- 핸들러 작성 (GET 요청의 URL의 쿼리스트링에서 data 추출)

```
73 # 경로의 시작이 ('/get/id') 인 경우 (경로의 끝에 쿼리스트링 추가되어 있음)
74 if self.path.startswith('/get/id'):
75     # URL(/get/id?key=value)에서 쿼리 스트링 파싱(분리)
76     url = parse.urlparse(self.path) # URL을 구성요소별 분리
77     q_string = url.query # 쿼리스트링의 폼 데이터 저장(key=value)
78     # parse_qs() : 폼데이터는 기본적인 문자열 --> 딕셔너리로 변환
79     params= parse.parse_qs(q_string)
80     print(f"파라미터 딕셔너리변환: {params}")
81     name = params.get('name',[''])[0] # name의 첫번째 값 추출
82     print(f"name= {name}")
83
84     # 추출한 name값을 포함해서 응답 페이지 보내기
85     self.send_response(200) # status line: HTTP/1.1 200 OK
86     self.send_header("Content-type", "text/html; charset=utf-8") #바디 type지정
87     self.end_headers() # 더이상 헤더없음. blank line 추가
88     # 응답메시지의 바디 전송
89     response_body = f"<h1>HTTP Server에 오신것을 환영합니다</h1><h2>안녕하세요, {name}님</h2>"
90     self.wfile.write(response_body.encode('utf-8'))
91
```

HTTP Server에 오신것을 환영합니다
안녕하세요, 홍길동님

3. http.server 모듈을 이용한 GET, POST 메소드 구현

■ 실습2) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍

- 핸들러 작성 (POST 요청의 body 폼데이터에서 data 추출), 서버 구동

```
92 def do_POST(self): # 빈 로직 상속받아 메소드 오버라이딩으로 자체 구현
93     print("POST 요청 들어옴")
94     # 경로의 끝이 루트('/post/id') 인 경우
95     if self.path.endswith('/post/id'):
96         # 웹브라우저 요청메시지 바디의 폼데이터(파라미터) 저장
97         content_length = int(self.headers['Content-length']) # 바디 길이저장
98         data = self.rfile.read(content_length).decode('utf-8') # 바디 폼데이터 저장
99         if data is not None:
100             # parse_qs() : 폼데이터는 기본적인 문자열 --> 딕셔너리로 변환
101             dic_data = parse.parse_qs(data)
102             print(f"폼데이터 딕셔너리변환: {dic_data}")
103             name = dic_data.get('name',[''])[0] # name의 첫번째 값 추출
104
105             # 바디에서 추출한 name을 포함해서 응답 페이지 보내기
106             self.send_response(200) # status line: HTTP/1.1 200 OK
107             self.send_header("Content-type", "text/html; charset=utf-8") #바디 type지정
108             self.end_headers() # 더이상 헤더없음. blank line 추가
109             # 응답메시지의 바디 전송
110             response_body = f"<h1>HTTP Server에 오신것을 환영합니다</h1><h2>안녕하세요, {name}님</h2>"
111             self.wfile.write(response_body.encode('utf-8'))
112
```

HTTP Server에 오신것을 환영합니다

안녕하세요, 홍길동님

3. http.server 모듈을 이용한 GET, POST 메소드 구현

- 실습2) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍
 - HTTP 웹 서버 생성 및 시작

```
113 if __name__ == "__main__":  
114     address = ('localhost', 8088)  
115     # HTTP 서버 생성 , HTTPServer(주소, 핸들러)  
116     server = HTTPServer(address, myHandler)  
117     print(f"웹 서버가 {address}로 서비스 되고 있습니다")  
118     server.serve_forever()    # 서버 시작(looping)  
119
```

4. 응용실습) http.server 모듈을 이용한 BMI 측정 웹 서버 구현

■ 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

- BMI(체질량지수)를 계산해 주는 웹 서버를 구현해 봅시다

- 사용자의 키(Cm)와 몸무게(Kg)을 입력받아 체질량지수를 계산해 주는 웹 서버 제작

GET 방식으로 파라미터(키, 몸무게) 전송하기

height(Cm)

weight(Kg)

`bmi = round(weight/(height/100)**2, 2)` #소수점 2자리까지만 출력

당신의 체질량지수는 20.76 입니다

4. 응용실습) http.server 모듈을 이용한 BMI 측정 웹 서버 구현

■ 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

GET방식- URL 쿼리스트링으로 전송

http://localhost:8089/get/bmi?height=170&weight=60

홈 페이지 (/index.html)
http://localhost:8089/

BMI(체질량지수) Calculator Welcome Page

GET 방식으로 파라미터 전달

POST 방식으로 파라미터 전달

GET 방식으로 파라미터(키, 몸무게) 전송하기

170

60

전송하기

POST방식- 바디 폼데이터 전송

http://localhost:8089/post/bmi

POST 방식으로 파라미터(키, 몸무게) 전송하기

170

60

전송하기

height=170&weight=60

응답 페이지(html)

당신의 체질량지수는 20.76 입니다

4. 응용실습) http.server 모듈을 이용한 BMI 측정 웹 서버 구현

- 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

파일명: httpServer_bmi_이름.py

```
1 # http.server 모듈을 이용한 BMI 측정 웹 서버 구현(GET, POST방식처리)
2 from http.server import HTTPServer, BaseHTTPRequestHandler
3
4 # URL 구성요소별 파싱하는 모듈(URL:프로토콜, 도메인, 파라미터(쿼리))
5 from urllib import parse
6
7 # 웹 서버 를 페이지 제작(첫 화면)
8 html_index = '''
9     <!doctype html>
10    <html>
11        <body>
12            <h1>BMI(체질량지수) Calculator Welcome Page</h1>
13            <h3><a href="/get">GET 방식으로 파라미터 전달</a></h3>
14            <h3><a href="/post">POST 방식으로 파라미터 전달</a></h3>
15        </body>
16    </html>
17 '''
```

BMI(체질량지수) Calculator Welcome Page

[GET 방식으로 파라미터 전달](#)

[POST 방식으로 파라미터 전달](#)

4. 응용실습) http.server 모듈을 이용한 BMI 측정 웹 서버 구현

■ 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

```
18 # GET 요청으로 파라미터를 전송하는 웹 페이지 제작
19 html_get = '''
20     <!doctype html>
21     <html>
22         <body>
23             <h1>GET 방식으로 파라미터(키, 몸무게) 전송하기</h1>
24             <form method="GET" action="/get/bmi">
25                 <p><input type="text" name="height" placeholder="키(Cm)를 입력하세요"></p>
26                 <p><input type="text" name="weight" placeholder="몸무게(Kg)을 입력하세요"></p>
27                 <p><input type="submit" value="전송하기"></p>
28             </form>
29         </body>
30     </html>
31 ...
32 # POST 요청으로 파라미터를 전송하는 웹 페이지 제작
33 html_post = '''
34     <!doctype html>
35     <html>
36         <body>
37             <h1>POST 방식으로 파라미터(키, 몸무게) 전송하기</h1>
38             <form method="POST" action="/post/bmi">
39                 <p><input type="text" name="height" placeholder="키(Cm)를 입력하세요"></p>
40                 <p><input type="text" name="weight" placeholder="몸무게(Kg)을 입력하세요"></p>
41                 <p><input type="submit" value="전송하기"></p>
42             </form>
43         </body>
44     </html>
45 ...
46
```

GET 방식으로 파라미터(키, 몸무게) 전송하기

POST 방식으로 파라미터(키, 몸무게) 전송하기

4. 응용실습) http.server 모듈을 이용한 BMI 측정 웹 서버 구현

■ 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

```
47 # base핸들러 상속받아 자체 핸들러 제작
48 class myHandler(BaseHTTPRequestHandler):
49     def do_GET(self): # 빈 로직 상속받아 메소드 오버라이딩으로 자체 구현
50         print("GET 요청 들어옴")
```

GET 방식에 대한 처리 로직을 구현해 보세요.

예) height=170&weight=60

당신의 체질량지수는 20.76 입니다

```
56 def do_POST(self): # 빈 로직 상속받아 메소드 오버라이딩으로 자체 구현
57     print("POST 요청 들어옴")
```

POST 방식에 대한 처리 로직을 구현해 보세요.

예) height=170&weight=60

당신의 체질량지수는 20.76 입니다

4. 응용실습) http.server 모듈을 이용한 BMI 측정 웹 서버 구현

- 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

```
120 if __name__ == "__main__":  
121     address = ('localhost', 8089)  
122     # HTTP 서버 생성 , HTTPServer(주소, 핸들러)  
123     server = HTTPServer(address, myHandler)  
124     print(f" BMI 측정 웹 서버가 {address}로 서비스 되고 있습니다")  
125     server.serve_forever()    # 서버 시작(looping)
```

5. (심화) http.server 모듈을 이용한 GET, POST 메소드 구현

■ 심회실습) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍

- Web의 구성요소를 POST 방식으로 받아서 웹 브라우저에 출력시켜 보세요
- 초기 홈페이지에는 1가지만 출력되어 있음

http://localhost:8088/

홈 페이지(/index.html)

Web 구성요소 4가지

- 웹 서버와 웹 브라우저

[웹 구성요소 추가하기](#)

웹의 세계에 오신것을 환영합니다

http.server를 이용해 웹서버를 제작해 보시다!

http://localhost:8088/new
POST 요청

웹 구성요소 추가하기

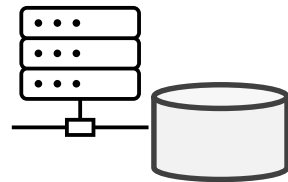
Web 구성요소 4가지

- 웹 서버와 웹 브라우저
- HTML

[웹 구성요소 추가하기](#)

웹의 세계에 오신것을 환영합니다

http.server를 이용해 웹서버를 제작해 보시다!



5. (심화) http.server 모듈을 이용한 GET, POST 메소드 구현

- 심화실습) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍

파일명: httpServer_adv.py

```
# http Server programming with baseHttpServer

from http.server import HTTPServer, BaseHTTPRequestHandler
# URL 구성요소별 파싱하는 모듈
import urllib.parse as parser

IP = 'localhost'
PORT = 8080
address = (IP, PORT)
```

5. (심화) http.server 모듈을 이용한 GET, POST 메소드 구현

■ 실습3) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍

- 루트('/') 경로 시 응답할 기본 홈 페이지(index.html)

```
elements = ['웹서버와 웹브라우저']

def html_get():
    liTags = ''
    for element in elements:
        liTags = liTags + f"<li>{element}</li>"
    return f'''<!doctype html>
<html>
    <body>
        <h1> Web의 구성요소 4가지</h1>
        <ul>
            {liTags}
        </ul>
        <h3><a href="/new">웹 구성요소 추가하기</a></h3>
        <h2>웹의 세계에 오신 것을 환영합니다</h2>
        웹서버를 제작해 봅시다!
    </body>
</html>
...'''
```

Web 구성요소 4가지

- 웹 서버와 웹 브라우저

웹 구성요소 추가하기

웹의 세계에 오신것을 환영합니다

http.server를 이용해 웹서버를 제작해 봅시다!

5. (심화) http.server 모듈을 이용한 GET, POST 메소드 구현

■ 실습3) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍

- '/new' 경로 시 응답할 웹 페이지(html_post.html)

```
html_post = '''
    <!doctype html>
    <html>
        <body>
            <h1> 웹 구성요소 추가하기</h1>
            <form method="POST" action="/new">
                <input name="item" type="text" placeholder="add new item">
                <input type="submit" value="제출하기">
            </form>
        </body>
    </html>
    ...
'''
```

웹 구성요소 추가하기

5. (심화) http.server 모듈을 이용한 GET, POST 메소드 구현

■ 실습3) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍

- 핸들러 : GET 요청 시 서버측 처리 코드

```
class myHTTPServerHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        print('GET 요청 들어옴')
        if self.path.endswith('/'): # 경로의 끝이 '/'인 경우, 기본적인 요청
            self.send_response(200)
            self.send_header('Content-Type', 'text/html; charset=utf-8')
            self.end_headers()

            self.wfile.write(html_get().encode('utf-8'))

        if self.path.endswith('/new'): # 경로의 끝이 '/new'인 경우
            self.send_response(200)
            self.send_header('Content-Type', 'text/html; charset=utf-8')
            self.end_headers()

            self.wfile.write(html_post.encode('utf-8'))
```

5. (심화) http.server 모듈을 이용한 GET, POST 메소드 구현

■ 실습3) HTTP 서버 모듈을 이용한 GET, POST 메소드 구현 프로그래밍

- 핸들러 : POST 요청 시 서버측 처리 코드 (폼데이터에서 item 값 추출하기)

```
def do_POST(self):
    print('POST 요청 들어옴')
    if self.path.endswith('/new'):
        # 클라이언트에서 요청한 바디 저장
        data = self.rfile.read(int(self.headers['content-length']))
        if data is not None:
            # query string 파싱, {key: [value1, value2, ... ]} 형태로 파싱
            data = dict(parser.parse_qs(data.decode('utf-8')))
            print(f'POST = {data}')

            elements.append(data['item'][0])

        self.send_response(200)
        self.send_header('Content-Type', 'text/html; charset=utf-8')
        self.end_headers()

        self.wfile.write(html_get().encode('utf-8'))
```

```
server = HTTPServer(address, myHTTPServerHandler)
print(f"웹 서버가 {PORT}으로 서비스 되고 있습니다")
server.serve_forever()
```



10주차 과제 안내

- 과제명: http.server 모듈을 이용한 GET, POST 메소드 구현 프로그래밍
 - 10주차에 응용실습한 BMI 측정 웹 서버 파이썬 파일을 CTL 과제 제출란에 제출
 - 첨부파일: httpServer_bmi_이름.py
 - 프로그램 상 오류 혹은 이해가 안 되는 부분은 comment 남겨 주세요.

BMI(체질량지수) Calculator Welcome Page

[GET 방식으로 파라미터 전달](#)

[POST 방식으로 파라미터 전달](#)

GET 방식으로 파라미터(키, 몸무게) 전송하기

POST 방식으로 파라미터(키, 몸무게) 전송하기

당신의 체질량지수는 20.76 입니다



Thank You
